

Unit 6: Polymorphism and Symbolic Computing

Table of contents

1	Running the codebase	1
2	Symbolic Computation	2
2.1	Elementary Algebra	2
2.1.1	Definition (Ring)	2
2.1.2	Definition (Zero Divisor)	2
2.1.3	Definition (Integral Domain)	2
2.1.4	Definition (Field)	3
2.1.5	Theorem (Division Algorithm)	3
2.2	Elementary Number Theory	4
2.2.1	Residue Classes	4
2.2.2	Symmetric Mod	5
2.2.3	Greatest Common Divisor	6
3	Examples	8
3.0.1	Inversion in $\mathbb{Z}_m$	9
3.1	The Polynomial Ring $\mathcal{R}[x]$	9
3.1.1	Definition	9
3.1.2	Representation of Polynomials in this project	9
3.1.3	Addition in $\mathcal{R}[x]$	11
3.1.4	Multiplication in $\mathcal{R}[x]$	12
3.1.5	Division in $\mathbb{F}[x]$	13
3.1.6	GCDs in $\mathbb{Z}_p[x]$	16

1 Running the codebase

Let's prepare by running the codebase for the computer algebra project:

```
using Pkg;
cd("../MATH2504_2026_ProjectB");
Pkg.activate(".");
Pkg.instantiate();
include("../MATH2504_2026_ProjectB/poly_factorization_project.jl"); # load/run the functions and
types of the project codebase
```

Activating project at `~/github/MATH2504\_2026\_ProjectB`

Now we have access to basic polynomials and algebraic operations acting on polynomials:

```
x = x_poly(PolynomialDense);
z = zero(PolynomialDense);
p=(3x^2-x)*x
```

-1 x^2 + 3 x^3

2 Symbolic Computation

To compute *exactly* means no error is introduced during arithmetic — something *not* true when working with floats. In a computer algebra system $\sqrt{2} = \sqrt{2}$ and not $\sqrt{2} = 1.41 \dots$ as one may expect.

Our interest is performing *exact* arithmetic in rings with efficient algorithms. In particular we are interested in the *ring of integers*

$$\mathbb{Z} = \{0, -1, 1, 2, -2, 3, -3, \dots\}$$

and *polynomials*

$$\mathcal{R}[x] = \left\{ \sum_{k=0}^N a_k x^k \text{ s.t. } k \in \mathbb{N}, a_k \in \mathcal{R} \right\}.$$

2.1 Elementary Algebra

It is first necessary to establish terms and definitions.

Let $\mathbb{N} = \{0, 1, 2, \dots\}$ be the set of *natural numbers*, $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ be the *integers* and $\mathbb{P}$ be the set of odd primes.

2.1.1 Definition (Ring)

A ring $\mathcal{R}$ is a set with addition (+) and multiplication ($\cdot$) satisfying the following conditions called the *ring axioms*:

- $(\mathcal{R}, +)$ is an abelian group, that is:
 - (+) is associative *and* commutative,
 - $\exists b \in \mathcal{R}$ called the *additive identity* satisfying $\forall a \in \mathcal{R} \ a + b = a$,
 - $\forall a \in \mathcal{R} \ \exists b \in \mathcal{R}$ called the *additive inverse* satisfying $a + b = 0$.
- $(\mathcal{R}, \cdot)$ is a monoid, that is:
 - ($\cdot$) is associative, and
 - $\exists b \in \mathcal{R}$ called the *multiplicative identity* satisfying $\forall a \in \mathcal{R}; \ a \cdot b = b \cdot a = a$
- Multiplication *distributes* over addition. That is, $\forall a, b, c \in \mathcal{R}$:
 - $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$
 - $(b + c) \cdot a = (b \cdot a) + (c \cdot a)$

Examples

1. The $\mathcal{R} = \{0\}$ (the *zero ring*) is a trivial ring.
2. The naturals $\mathbb{N}$ do *not* form a ring as they lack additive identity.
3. Arithmetic on the clock forms a ring.

2.1.2 Definition (Zero Divisor)

An element $a \in \mathcal{R}$ is called a *zero divisor* if there is nonzero $b \in \mathcal{R}$ such that $ab = 0$.

2.1.3 Definition (Integral Domain)

An integral domain is a nonzero commutative ring without zero divisors.

Examples

1. The integers $\mathbb{Z}$ form an integral domain.
2. ‘Clock arithmetic’ does not form an integral domain because, for instance, $3 \cdot 4$ is 12 which is the zero.

2.1.4 Definition (Field)

A ring $\mathcal{R}$ is also a *field* when each non-zero element has a *multiplicative inverse*. Equivalently, $\mathcal{R}$ is a ring when

$$\forall a \in \mathcal{R}^{\neq 0}; \exists b \in \mathcal{R} : a \cdot b.$$

Examples

For instance, the *rationals*

$$\mathbb{Q} = \left\{ \frac{a}{b} : a, b \in \mathbb{Z}, b \neq 0 \right\}$$

is a field because $\frac{a}{b} \cdot \frac{b}{a} = 1$ when $a, b \neq 0$.

2.1.5 Theorem (Division Algorithm)

Let $a, b \in \mathbb{Z}$ with $b > 0$. There is *unique* q and r (called the *quotient* and *remainder*) satisfying

$$a = b \cdot q + r$$

with $r < |q|$.

```
# An example
a, b = 23, 7
```

```
(23, 7)
```

```
q = a ÷ b # \div + [TAB]
```

```
3
```

```
r = a % b
```

```
2
```

```
a == b * q + r
```

```
true
```

```
# Recall: abstract type «name» <: «supertype» end
function remainder(a::T, b::T) where T <: Integer
    #Note that later we'll extend this to integral domains

    #short circuit evaluation gives cheeky if statements
    a < 0 && return remainder(-a, b) #later replace `0` with `zero(T)`
    a < b && return a
    return remainder(a-b, b)
end
```

```
remainder (generic function with 1 method)
```

```
#note there is an in-built `rem`
remainder(23, 7), rem(23, 7), 23 % 7, 23 - (23 ÷ 7)*7
```

```
(2, 2, 2, 2)
```

```
remainder(15, 5), remainder(4, 101), remainder(125, 4)
```

```
(0, 4, 1)
```

```
function quo(a::T, b::T) where T <: Integer
    a < 0 && return -quo(-a,b)
    a < b && return 0
    return 1 + quo(a-b, b)
end
```

quo (generic function with 2 methods)

```
quo(15, 5), quo(4, 101), quo(125, 4)
```

```
(3, 0, 31)
```

```
for (a,b) in [(15, 5), (4, 101), (125,4 )]
    q, r = quo(a, b), rem(a, b)
    println("$a = $q $b + $r")
end
```

```
15 = 3 5 + 0
```

```
4 = 0 101 + 4
```

```
125 = 31 4 + 1
```

Of course, in Julia we have % and ÷ for remainder and and quotient.

(Note that in Python // is the quotient whereas in Julia // defines a rational type).

```
23 // 7, float(23//7) #The first // is a rational type #!!! Note in Python // is ÷
```

```
(23//7, 3.2857142857142856)
```

Incidentally, another way to express the quotient/remainder relation is by way of “proper” fractions.

$$23 = 3 \cdot 7 + 2 \iff \frac{23}{7} = 3 + \frac{2}{7}$$

```
(23 ÷ 7), (23 % 7) // 7
```

```
(3, 2//7)
```

```
23 // 7 == (23 ÷ 7) + (23 % 7) // 7
```

```
true
```

2.2 Elementary Number Theory

2.2.1 Residue Classes

The sets $\mathbb{Z}_n = \{0, 1, \dots, n-1\}$ for $n \in \mathbb{N}$ are rings called *residue classes* when addition and multiplication are done ‘mod n’. This is sometimes called ‘clock arithmetic’ as three hours past eleven is two because $11 + 3 \cong 2 \pmod{12}$.

We say/write

$$a \cong b \pmod{c}$$

when *a is congruent to b modulo c*. Also:

$$a \cong b \pmod{c} \iff \text{rem}(a, c) = b \iff a \% c = b$$

We have already seen something similar with overflow. In particular, working in an 8-bit register is essentially doing arithmetic modulo 2^8 .

```
UInt8(28-3) + UInt8(4)
```

```
0x01
```

```
(28-3 + 4) % 28
```

```
1
```

2.2.2 Symmetric Mod

We may also use negatives values, here $\mathbb{Z}_n = \{-\text{quo}(n,2), 0, \dots, \text{quo}(n,2)\}$. For instance, $\mathbb{Z}_7 = \{-3, -2, -1, 0, 1, 2, 3\}$ and

$$5 \cong -2 \text{ smod } 7.$$

We do not use this notation and simply move use `mod` as `smod` when we want as there is no effect on the theory and presentation.

Example (Addition Table)

$\mathbb{Z}_6 = \{0, \dots, 5\}$ has the following *addition table*

```
n = 6
Z_n = 0:(n-1)
typeof(Z_n)
A = [(x+y) % n for y in Z_n, x in Z_n]
```

```
6×6 Matrix{Int64}:
```

```
0  1  2  3  4  5
1  2  3  4  5  0
2  3  4  5  0  1
3  4  5  0  1  2
4  5  0  1  2  3
5  0  1  2  3  4
```

Notice: 1. the addition table is symmetric (equal to its transpose) which can only happen when the addition is commutative, 1. the additive identity is 0, 1. each column (and row) has 0 and thereby each element has an additive inverse: for instance, the third column indicates that 2 has additive inverse 4 and correspondingly $2 + 4 \cong 0 \text{ mod } 6$.

```
(A .== 0)
```

```
6×6 BitMatrix:
```

```
1  0  0  0  0  0
0  0  0  0  0  1
0  0  0  0  1  0
0  0  0  1  0  0
0  0  1  0  0  0
0  1  0  0  0  0
```

```
additive_inverses = [findfirst((A .== 0)[: ,k+1])-1 for k in Z_n]
println(additive_inverses) # print horizontally instead of vertically
```

```
[0, 5, 4, 3, 2, 1]
```

Example (Multiplication Table)

$\mathbb{Z}_6$ has the following *multiplication table*

```
M = [(x*y) % n for y in Z_n, x in Z_n]
```

```
6×6 Matrix{Int64}:
```

```
0 0 0 0 0 0
0 1 2 3 4 5
0 2 4 0 2 4
0 3 0 3 0 3
0 4 2 0 4 2
0 5 4 3 2 1
```

Notice: 1. the multiplicative identity is 1, 1. *not* all rows contain one, which means there are some elements that do *not* have *multiplicative inverse*. For instance there is no $b \in \mathbb{Z}_6$ such that $2 \cdot b \cong 1 \pmod{6}$.

```
println([(2*y) % n for y in Z_n])
```

```
[0, 2, 4, 0, 2, 4]
```

```
mult_inverses = Z_n[[sum((M .== 1)[: ,k+1]) > 0 for k in Z_n]]
println("Elements with a multiplicative inverse: ", mult_inverses )
println("Elements without a multiplicative inverse: ", setdiff(Z_n,mult_inverses))
```

```
Elements with a multiplicative inverse: [1, 5]
Elements without a multiplicative inverse: [0, 2, 3, 4]
```

Notice $\mathbb{Z}_6$ is *not* a field because, recall, there is no multiplicative inverse for 2. However, $\mathbb{Z}_p$ is a field when p is a prime number.

Example

$\mathbb{Z}_5 = \{0, \dots, 4\}$ is a field and has the following multiplication table.

```
n = 5
Z_n = 0:(n-1)
M = [(x*y) % n for y in Z_n, x in Z_n]
```

```
5×5 Matrix{Int64}:
```

```
0 0 0 0 0
0 1 2 3 4
0 2 4 1 3
0 3 1 4 2
0 4 3 2 1
```

```
mult_inverses = Z_n[[sum((M .== 1)[: ,k+1]) > 0 for k in Z_n]]
println("Elements with a multiplicative inverse: ", mult_inverses )
println("Elements without a multiplicative inverse: ", setdiff(Z_n,mult_inverses))
```

```
Elements with a multiplicative inverse: [1, 2, 3, 4]
Elements without a multiplicative inverse: [0]
```

Notice every column (and row) contains 1 asides the first column (and row) which corresponds to 0. For instance, the third column indicates the inverse of 3 is 2 and correspondingly $3 \cdot 2 \cong 1 \pmod{5}$.

2.2.3 Greatest Common Divisor

The *greatest common divisor* is one of the fundamental operations of a computer algebra system. For instance, we need `gcd` to reduce fractions to their canonical form.

2.2.3.1 Def (GCD)

Let $a, b \in \mathbb{Z}$ not both 0. We say $g \in \mathbb{Z}$ is the *greatest common divisor* of a and b , denoted $\gcd(a, b)$, when 1. $g|a$ and $g|b$ (g is a common divisor), 1. $h|a \wedge h|b \implies h|g$ (greatest), 1. $g > 0$ (required for uniqueness)

Example (GCD)

- The $\gcd(6, 4) = \gcd(2 \cdot 3, 2 \cdot 2) = 2$.
- The $\gcd(6, 0) = \gcd(1 \cdot 6, 0 \cdot 6) = 6$.

```
#The is an in-built gcd() function for integers (and a few more types)
#but we'll soon make our own
using Random; Random.seed!(10)

a_test = *(rand(1:100,5)...)
b_test = *(rand(1:100,5)...)
@show a_test, b_test
Base.gcd(a_test, b_test)
```

```
(a_test, b_test) = (4145382, 1056517560)
```

```
18
```

2.2.3.2 Euclid's Algorithm

The **Euclidean Algorithm** computes the gcd of two integers. (Actually the Euclidean Algorithm computes gcds for any two elements of a *Euclidean Domain*). The algorithm exploits the following property of the gcd:

Lemma

Let $a, b \in \mathbb{Z}^{>0}$ and $a = b \cdot q + r$ with $r \in [0, b)$. Then 1. $\gcd(b, a) = \gcd(a, b)$, 1. $\gcd(a, b) = \gcd(r, b)$, 1. $\gcd(a, b) = \gcd(a - b, b)$.

It is straightforward to turn this lemma into an algorithm:

```
function euclid_alg(a,b)
    (b == 0) && return a
    return euclid_alg(b, a % b) # Rule 1 and 2
end

euclid_alg(2, 2*2), euclid_alg(3*7, 2*3), euclid_alg(2*2, 13)
```

```
(2, 3, 1)
```

```
Base.gcd(a_test, b_test), euclid_alg(a_test,b_test)
```

```
(18, 18)
```

2.2.3.2.1 Extended Euclid's Algorithm

The **Extended Euclidean Algorithm** in addition to the $\gcd(a, b)$, also computes s and t (called the Bézout coefficients) such that

$$as + bt = \gcd(a, b).$$

```
function i_ext_euclid_alg(a,b)
    (a == 0) && return b, 0, 1
    g, s, t = i_ext_euclid_alg(b % a, a)
```

```

    return g, t - (b ÷ a)*s, s
end

pretty_print_egcd((a,b),(g,s,t)) = println("$a×$s + $b×$t = $g = gcd($a, $b)") #\times + [TAB]

for (a,b) in [(4,12), (9,12), (4,13)]
    pretty_print_egcd((a,b), i_ext_euclid_alg(a,b))
end

```

```

4×1 + 12×0 = 4 = gcd(4, 12)
9×-1 + 12×1 = 3 = gcd(9, 12)
4×-3 + 13×1 = 1 = gcd(4, 13)

```

```

#Note there is an in-built gcdx - let's compare against that too
Base.gcdx(a_test,b_test), i_ext_euclid_alg(a_test,b_test)

```

```
((18, 2477299, -9720), (18, 2477299, -9720))
```

```
?gcdx
```

```
search: gcdx gcd ccdf logcdf cd ecdf cdf
```

```
gcdx(a, b...)
```

Computes the greatest common (positive) divisor of a and b and their Bézout coefficients, i.e. the integer coefficients u and v that satisfy $u * a + v * b = d = \gcd(a, b)$. `gcdx(a, b)` returns (d, u, v) .

For more arguments than two, i.e., `gcdx(a, b, c, ...)` the Bézout coefficients are computed recursively, returning a solution $(d, u, v, w, \dots)$ to $u * a + v * b + w * c + \dots = d = \gcd(a, b, c, \dots)$.

The arguments may be integer and rational numbers.

!!! compat “Julia 1.4” Rational arguments require Julia 1.4 or later.

!!! compat “Julia 1.12” More or fewer arguments than two require Julia 1.12 or later.

3 Examples

```

julia> gcdx(12, 42)
(6, -3, 1)

julia> gcdx(240, 46)
(2, -9, 47)

julia> gcdx(15, 12, 20)
(1, 7, -7, -1)

```

!!! note Bézout coefficients are *not* uniquely defined. `gcdx` returns the minimal Bézout coefficients that are computed by the extended Euclidean algorithm. (Ref: D. Knuth, TAOCP, 2/e, p. 325, Algorithm X.) For signed integers, these coefficients u and v are minimal in the sense that $|u| < |b/d|$ and $|v| < |a/d|$. Furthermore, the signs of u and v are chosen so that d is positive. For unsigned integers, the coefficients u and v might be near their `typemax`, and the identity then holds only via the unsigned integers’ modulo arithmetic.

3.0.1 Inversion in $\mathbb{Z}_m$

Notice the Extended Euclidean Algorithm computes inverses in $\mathbb{Z}_m$. Given $a \in \mathbb{Z}_m^{\neq 0}$ the `egcd(a, m)` returns s and t such that

$$a \cdot s + m \cdot t = \gcd(a, m).$$

Taking the entire equation $\pmod m$ we get

$$\begin{aligned} \gcd(a, m) &\cong a \cdot s + m \cdot t \pmod m \\ \implies \gcd(a, m) &\cong a \cdot s + 0 \pmod m \\ \implies \gcd(a, m) &\cong a \cdot s \pmod m. \end{aligned}$$

Provided $\gcd(a, m) = 1$ (i.e. “coprime” or “relatively prime”) we get $a \cdot s \cong 1 \pmod m$ and thus $a^{-1} \cong s \pmod m$.

Example

To find the inverse of $5 \in \mathbb{Z}_{13}$ do:

```
a, m = 5, 13
i_ext_euclid_alg(5, 13)
```

```
(1, -5, 2)
```

```
inverse_mod(a,m) = mod(i_ext_euclid_alg(a,m)[2], m)
```

```
inverse_mod (generic function with 1 method)
```

```
i = inverse_mod(a,m)
```

```
8
```

```
mod(a*i, m)
```

```
1
```

3.1 The Polynomial Ring $\mathcal{R}[x]$

By this point in your mathematics career you have surely worked with *polynomials*. A polynomial is something like $x^2 + 2x + 1$ but *not* like $\frac{x^2+1}{x-1}$ or $x \sin(x)$.

3.1.1 Definition

Let $\mathcal{R}$ be a ring and $a \in \mathcal{R}[x]$. Let

$$a = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

with $a_n \neq 0$.

- a is called a **polynomial** in x and $a_k x^k$ are its **terms**.
- The **degree** of a is n : $\deg(a) = n$.
- The **leading coefficient** of a is a_n : $\text{lc}(a) = a_n$.
- The **leading term** of a is $a_n x^n$: $\text{lt}(a) = a_n x^n$.

By convention we let $\text{lc}(0) = \text{lt}(0) = 0$ and $\deg(0) = -\infty$.

3.1.2 Representation of Polynomials in this project

```

```{julia}
"""
A term.
"""
struct Term #structs are immutable by default
 coeff::Int
 degree::Int
 function Term(coeff::Int, degree::Int)
 degree < 0 && error("Degree must be non-negative")
 coeff != 0 ? new(coeff,degree) : new(coeff,0)
 end
end

"""
Creates the zero term.
"""
zero(::Type{Term})::Term = Term(0,0)

"""
Creates the unit term.
"""
one(::Type{Term})::Term = Term(1,0)
```

```

x+x

2 x¹

```

```{julia}
"""
A Polynomial type - designed to be for polynomials with integer coefficients.
"""
struct Polynomial

 #A zero packed vector of terms
 #Terms are assumed to be in order
 #until the degree of the polynomial. The leading term (i.e. last) is assumed to be non-zero
 #except
 #for the zero polynomial where the vector is of length 1.
 #Note: at positions where the coefficient is 0, the power of the term is also 0 (this is how
 #the Term type is designed)
 #The maximum length allowed for the vector is max_degree+1
 terms::Vector{Term}

 #Inner constructor of 0 polynomial
 Polynomial() = new([zero(Term)])

 #Inner constructor of polynomial based on arbitrary list of terms
 function Polynomial(vt::Vector{Term})

 #Filter the vector so that there is not more than a single zero term
 vt = filter((t)->!iszero(t), vt)
 if isempty(vt)
 vt = [zero(Term)]
 end

 max_degree = maximum((t)->t.degree, vt)
 max_degree > max_degree_allowed && error("Cannot create polynomial of degree greater than
 $max_degree_allowed. You are trying $max_degree.")
 end
end
```

```

```

        terms = [zero(Term) for i in 0:max_degree] #First set all terms with zeros

        #now update based on the input terms
        for t in vt
            terms[t.degree + 1] = t #+1 accounts for 1-indexing
        end
        return new(terms)
    end
end

"""
Construct a polynomial of the form x.
"""
x_poly() = Polynomial(Term(1,1))

"""
Creates the zero polynomial.
"""
zero(::Type{Polynomial})::Polynomial = Polynomial()

"""
Creates the unit polynomial.
"""
one(::Type{Polynomial})::Polynomial = Polynomial(one(Term))
one(p::Polynomial) = one(typeof(p))
"""

```

3.1.3 Addition in $\mathcal{R}[x]$

For $\mathcal{R}[x]$ to be a *ring* we must define addition.

```

def ADDITION:
    Input    a, b in ZZ[x]
    Output   a + b (definitionally)

    if deg(a) == deg(b)
        term = lt(a) + lt(b)      # addition on *terms* ax^n + bx^n = (a+b)x^n
        return term + ADDITION(a-lt(a), b-lt(b))

    if deg(a) > deg(b)
        return lt(a) + ADDITION(a-lt(a), b)

    # deg(b) > deg(a)
    return lt(b) + ADDITION(a, b-lt(b))

```

```

"""{julia}
"""
Add a polynomial and an integer.
"""
+(p::Polynomial, n::Int) = p + Term(n,0)
+(n::Int, p::Polynomial) = p + Term(n,0)

"""
Add a polynomial and a term.
"""
function +(p::Polynomial, t::Term)
    p = deepcopy(p)
    if t.degree > degree(p)
        push!(p, t)
    end
end

```

```

else
    if !iszero(p.terms[t.degree + 1]) #+1 is due to indexing
        p.terms[t.degree + 1] += t
    else
        p.terms[t.degree + 1] = t
    end
end

return trim!(p)
end
+(t::Term, p::Polynomial) = p + t

"""
Add two polynomials. Does not assume dense (or spart) representation
"""
function +(p1::Polynomial, p2::Polynomial)::Polynomial
    p = deepcopy(p1)
    for t in p2
        p += t
    end
    return p
end
"""
"""

```

```

x = x_poly(PolynomialDense)
p1 = 2x^2 + 3x + (-5) #Project: Fix this so we can simply do 2x^2+3x-5
p2 = -3x^2 - 4x + 6
p1+p2

```

1 x⁰ + -1 x¹ + -1 x²

3.1.4 Multiplication in $\mathcal{R}[x]$

For $\mathcal{R}[x]$ to be a *ring* we must define multiplication.

```

def MULTIPLICATION:
    Input    a, b in ZZ[x]
    Output   a * b (definitionally)

    if a = 0 or b = 0:
        return 0

    if lt(a) = a and lt(b) = b:    # a and b are single terms
        return a * b                # product of *terms* ax^n * bx^m = (ab)x^(n+m)

    write a = a1 + a2 with deg(a1) != deg(a2)
    write b = b1 + b2 with deg(b1) != deg(b2)

    # DISTRIBUTION PROPERTY
    # (a1 + a2)*(b1 + b2) = a1*b1 + a1*b2 + a2*b1 + a2*b2
    return M(a1,b1) + M(a1,b2) + M(a2,b1) + M(a2,b2) where M = MULTIPLICATION

```

```

"""{julia}
"""
Multiplication of polynomial and term.
"""
*(t::Term, p1::Polynomial)::Polynomial = iszero(t) ? Polynomial() : Polynomial(map((pt)->t*pt,
p1.terms))
*(p1::Polynomial, t::Term)::Polynomial = t*p1

```

```

"""
Multiplication of polynomial and polynomial.
"""
function *(p1::Polynomial, p2::Polynomial)::Polynomial
    p_out = Polynomial()
    for t in p1
        p_out = p_out + (t * p2)
    end
    return p_out
end
"""

```

```

x = x_poly(PolynomialDense)
p1 = 2x^2 + 3x + (-5)
p2 = -3x^2 - 4x + 6
p1 * p2

```

```
-30 x^0 + 38 x^1 + 15 x^2 + -17 x^3 + -6 x^4
```

The following properties are consequences of our definition. (They are obligated to be).

3.1.4.0.1 Proposition (Polynomial Multiplication)

When $\mathcal{R}$ is an **integral domain** (a nonzero commutative ring in which the product of any two nonzero elements is nonzero) and $a, b \in \mathcal{R}[x]^{\neq 0}$ then 1. $\deg(ab) = \deg a + \deg b$, 1. $\text{lc}(ab) = a_b \cdot b_m = \text{lc}(a) \cdot \text{lc}(b)$, 1. $\text{lm}(ab) = \text{lm}(a) \cdot \text{lm}(b)$, 1. $\text{lt}(ab) = \text{lt}(a) \cdot \text{lt}(b)$.

Proof (Sketch)

$$\begin{aligned} a \times b &= (a_n x^n + \cdots + a_0)(b_m x^m + \cdots + b_0) \\ &= a_n \cdot b_m x^{n+m} + \cdots + a_0 \cdot b_0 \end{aligned}$$

3.1.4.0.2 Proposition

Every field is also an integral domain.

Proof (Omitted)

3.1.5 Division in $\mathbb{F}[x]$

We are unable to *divide* in $\mathcal{R}[x]$. Consider the following operation in $\mathbb{Z}[x]$

$$\frac{x^2}{2x} = \frac{1}{2}x.$$

Even though x^2 and $2x$ are from $\mathbb{Z}[x]$ that their division is in $\mathbb{Q}[x]$. This is because $\mathbb{Z}$ is not a *field* but $\mathbb{Q}$ is.

It turns out $\mathbb{F}[x]$ is a **Euclidean Domain** when $\mathbb{F}$ is a field. Re-consider the division, but this time let us do the calculation in $\mathbb{Z}_7[x]$

$$\begin{aligned} \frac{x^2}{2x} &\cong \frac{1}{2}x \pmod{7} \\ &\cong 2^{-1}x \pmod{7} \\ &\cong 4x \pmod{7}. \end{aligned}$$

Note that $\frac{1}{2} \cong 4 \pmod{7}$.

3.1.5.1 Theorem (Division in $\mathbb{F}[x]$)

Let $a, b \in \mathbb{F}[x]$, $b \neq 0$. There exists *unique* polynomials $q, r \in \mathbb{F}[x]$ such that

$$a = bq + r \quad \text{and} \quad r = 0 \quad \text{or} \quad \deg r < \deg b$$

Proof (Uniqueness) Let $a = bq_1 + r_1 = bq_2 + r_2$ with $\deg r_1, \deg r_2 < \deg b$.

Consider

$$\begin{aligned} bq_1 + r_1 &= bq_2 + r_2 \implies b(q_1 - q_2) = (r_2 - r_1) \\ &\implies b \mid (r_2 - r_1) && \text{But } \deg r_1, r_2 < \deg b \\ &\implies r_2 - r_1 = 0 \\ &\implies r_2 = r_1 \end{aligned}$$

Therefore $b(q_1 - q_2) = 0$. But since $\mathbb{F}[x]$ is an integral domain it cannot have zero divisors. As we assumed $b \neq 0$ this implies

$$(q_1 - q_2) = 0 \implies q_1 = q_2. \quad \square$$

Proof (Existence – The Division Algorithm)

```
def DIVISION:
  Input  a, b in ZZ_p[x] such that b != 0
  Output q, r such that a = b*q + r and deg r < deg b in ZZ_p[x]

  r, q = a, 0
  while r != 0 and deg r >= deg b do:
    t <- lt(r)/lt(b)    # division of *terms* in ZZ_p: (ax^n)/(bx^m) = a/b mod p x^(n-m), n>m
    q <- q + t
    r <- r - t*b        # deg r *must* monotonically decrease
  return (q,r)
```

A couple of convenience methods

```
def QUO(a, b) = first(DIVISION(a,b)) in ZZ_p[x]
def REM(a, b) = second(DIVISION(a,b)) in ZZ_p[x]
```

```
function div(t1::Term{C, D}, t2::Term{C, D}) where {C, D} # Base.:÷
  @assert t1.degree >= t2.degree
  return p->Term(mod(t1.coeff*int_inverse_mod(t2.coeff,p),p), t1.degree - t2.degree)
end
```

#At the moment, the divide function and it's friends (÷ and rem) return a function that then uses a prime to return an actual polynomial

```
function divide(num::Polynomial, den::Polynomial)
  function division_function(p::Int)
    f, g = mod(num,p), mod(den,p)
    degree(f) < degree(g) && return nothing
    iszero(g) && throw(DivideError())
    q = typeof(num)()
    prev_degree = degree(f)
    while degree(f) >= degree(g)
      h = typeof(num)( (leading(f) ÷ leading(g))(p) ) #synergy
      f = mod((f - h*g), p)
      q = mod((q + h), p)
      prev_degree == degree(f) && break
    prev_degree = degree(f)
  end
```

```

    end
    @assert iszero( mod((num - (q*g + f)),p))
    return q, f
end
return division_function
end

"""
The quotient from polynomial division. Returns a function of an integer.
"""
÷(num::Polynomial, den::Polynomial) = (p::Int) -> first(divide(num,den)(p))

"""
The remainder from polynomial division. Returns a function of an integer.
"""
rem(num::Polynomial, den::Polynomial) = (p::Int) -> last(divide(num,den)(p))

```

Base.rem

```

@show p1
@show p2
divide(p1, p2)

```

```

p1 = -5 x^0 + 3 x^1 + 2 x^2
p2 = 6 x^0 + -4 x^1 + -3 x^2

```

```

(::Main.Notebook.var"#division_function#divide##0"{Main.Notebook.PolynomialDense, Main.Notebook.PolynomialDense})

```

```

q , r = divide(p1,p2)(101)

```

```

(33 x^0, 100 x^0 + 34 x^1)

```

```

q = (p1 ÷ p2)(101) #Not the nicest interface, but will be nicer with PolynomialModP

```

```

33 x^0

```

```

r = rem(p1,p2)(101) #Not the nicest interface, but will be nicer with PolynomialModP

```

```

100 x^0 + 34 x^1

```

```

mod(q*p2+r - p1,101)

```

```

0

```

Example

Consider $a = 10x^2 + 4x + 1$ divided by $b = 5x - 3$ in $\mathbb{Z}_{11}[x]$.

$$r, q \leftarrow 10x^2 + 4x + 1, 0$$

$$r \neq 0 \text{ and } \deg r = 2 \geq \deg b = 1$$

$$t \leftarrow \frac{10x^2}{5x} \cong 2x \pmod{11}$$

$$q \leftarrow 0 + 2x$$

$$r \leftarrow (10x^2 + 4x + 1) - 2x(5x - 3) \cong 10x + 1 \pmod{11}$$

$$r \neq 0 \text{ and } \deg r = 1 \geq \deg b = 1$$

$$t \leftarrow \frac{10x}{5x} \cong 2 \pmod{11}$$

$$q \leftarrow 2x + 2$$

$$r \leftarrow (10x + 1) - 2(5x - 3) \cong 7 \pmod{11}$$

$$r \neq 0 \text{ and } \deg r = 0 \geq \deg b = 1$$

$$\text{return } (2x + 2, 7)$$

And notice $10x^2 + 4x + 1 \cong (5x - 3)(2x + 2) + 7 \pmod{11}$.

3.1.6 GCDs in $\mathbb{Z}_p[x]$

3.1.6.1 Theorem

Let $a, b \in \mathbb{Z}_p[x]^{\neq 0}$. There is $s, t \in \mathbb{Z}_p[x]$ such that

$$sa + tb = \gcd(a, b).$$

Proof: The extended Euclidean Algorithm

```
#Taken from polynomial_gcd.jl
#Note that as part of your project you'll implelment this for PolynomialModP
#(without an input argument `prime`)

function extended_euclid_alg(a::Polynomial, b::Polynomial, prime::Int)
    old_r, r = mod(a,prime), mod(b,prime)
    old_s, s = one(typeof(a)), zero(typeof(a))
    old_t, t = zero(typeof(a)), one(typeof(a))

    while !iszero(mod(r,prime))
        q = divide(old_r, r)(prime) |> first      # |> is the pipe operator
        old_r, r = r, mod(old_r - q*r, prime)
        old_s, s = s, mod(old_s - q*s, prime)
        old_t, t = t, mod(old_t - q*t, prime)
    end
    g, s, t = old_r, old_s, old_t
    @assert mod(s*a + t*b - g, prime) == 0
    return g, s, t
end
```

extended_euclid_alg (generic function with 2 methods)

```
x = x_poly(PolynomialDense)
p1 = 2x^2+3x +(-5) #Note we need +(-5)... try later without... how to fix?
```

```
p2 = -3x2 - 4x + 6
```

```
extended_euclid_alg(p1*p2, p2, 101)
```

```
(6 x0 + 97 x1 + 98 x2, 0, 1 x0)
```